Автономная некоммерческая организация высшего образования
«Университет Иннополис»

АННОТАЦИЯ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
(БАКАЛАВРСКУЮ РАБОТУ)
ПО НАПРАВЛЕНИЮ ПОДГОТОВКИ
09.03.01 «Информатика и вычислительная техника»

НАПРАВЛЕННОСТЬ (ПРОФИЛЬ) ОБРАЗОВАТЕЛЬНОЙ ПРОГРАММЫ
«Анализ данных и искусственный интеллект»

Тема	**Устранение разрыва в проверке обеспечения безопасности: обнаружение расхождений между заявленным и фактическим положением безопасности в контейнерных средах Kubernetes**

Выполнил	**Собех Йехия**	

Иннополис, 2026

АННОТАЦИЯ

Актуальность темы исследования

Современные системы активно используют микросервисную архитектуру на Kubernetes. По данным CNCF, более 60% организаций применяют Kubernetes в промышленной среде. Масштабное внедрение порождает проблемы безопасности: увеличение поверхности атаки, сложные цепочки зависимостей и высокая динамика жизненного цикла контейнеров.

Kubernetes обеспечивает безопасность через декларативные примитивы в контексте безопасности пода: seccomp, capabilities, ограничения пользователя, изоляция пространств имён. Экосистема располагает инструментами валидации (OPA Gatekeeper, Kyverno, PSA, Kubescape, Trivy Operator).

Однако все инструменты основаны на критическом допущении: если API сообщает о свойстве безопасности, среда выполнения его обеспечила. Данная работа демонстрирует, что это допущение не выполняется для привилегированных контейнеров.

Постановка проблемы

Трансляция спецификации пода через kubelet, CRI и OCI в примитивы ядра может молчаливо изменять ограничения безопасности. Данный разрыв верификации обеспечения означает отсутствие инструментов проверки фактического применения деклараций.

Оператор, задающий seccompProfile.type: RuntimeDefault для привилегированного контейнера, полагает, что фильтрация активна. Однако Docker молчаливо удаляет фильтр seccomp из привилегированных контейнеров, и ни один существующий инструмент не обнаруживает этого.

Цель и задачи исследования

Цель — проектирование, реализация и оценка инструмента верификации обеспечения контекста безопасности Kubernetes средой выполнения, а также характеризация категорий расхождений.

Задачи: (1) обзор литературы для характеризации разрыва верификации; (2) проектирование двухуровневой архитектуры по семи измерениям безопасности; (3) разработка алгоритма нормализации capabilities; (4) реализация уровня абстракции для Docker и containerd; (5) развёртывание и оценка на семи тестовых подах и четырёх компонентах плоскости управления; (6) классификация категорий расхождений.

Объект и предмет исследования

Объектом исследования является цепочка обеспечения безопасности контейнеризированных рабочих нагрузок в Kubernetes — от API через среду выполнения до примитивов ядра Linux.

Предметом являются расхождения между декларированным и фактически обеспечиваемым состоянием безопасности, а также методы их обнаружения.

Научная новизна

Впервые идентифицирован разрыв верификации обеспечения в Kubernetes. Предложена двухуровневая архитектура с попольным сравнением API и среды выполнения по семи измерениям.

Продемонстрировано: (i) Docker удаляет seccomp из привилегированных контейнеров; (ii) docker inspect опускает неявные профили. Предложен алгоритм нормализации capabilities.

Методология исследования

Разработанный инструмент k8s-psa (Kubernetes Pod Security Auditor) развёртывается как DaemonSet Kubernetes, обеспечивая работу сканирующего пода на каждом узле кластера.

Инструмент реализует двухуровневую архитектуру. Уровень 1 оценивает спецификации подов по восьми политикам безопасности (привилегированный режим, root, пространства имён хоста, seccomp, capabilities, read-only rootfs), предоставляя функциональность контроллеров допуска.

Уровень 2 (ключевой вклад) строит для каждого контейнера SpecSecurityState и RuntimeSecurityState, сравнивая их по семи измерениям. Несоответствия классифицируются: CRITICAL, WARNING или INFO.

Уровень абстракции скрывает специфику Docker и containerd за единым интерфейсом RuntimeSecurityState.

Алгоритм нормализации вычисляет ожидаемые capabilities из 14 по умолчанию с учётом списков add/drop, устраняя ложноположительные результаты.

Экспериментальная среда и оценка

Все эксперименты проведены на одноузловом кластере Minikube (Ubuntu 22.04 LTS, Kubernetes v1.32.0, Docker Engine 27.4.1 через cri-dockerd). Данные собраны за три полных цикла сканирования с интервалом 60 секунд.

Спроектировано семь тестовых подов: (1) полностью защищённый (контроль); (2) без контекста безопасности; (3) привилегированный с seccomp

(ключевой сценарий); (4) привилегированный без seccomp (контроль для пода 3); (5) частичные настройки; (6) запрет эскалации привилегий; (7) пространства имён хоста. Дополнительно использованы четыре компонента плоскости управления Kubernetes.

Основные результаты

Результаты оценки выявили три категории расхождений. **Первая категория — истинный разрыв обеспечения (CRITICAL).** Привилегированные контейнеры с seccompProfile.type: RuntimeDefault имеют фильтр seccomp молчаливо удалённым Docker. Под 3 продемонстрировал критическое расхождение: спецификация декларирует RuntimeDefault, среда выполнения — unconfined.

Вторая категория — инфляция capabilities (WARNING). Привилегированный режим предоставляет все 41 capability (на 27 больше по умолчанию). Инструмент зафиксировал 27 предупреждений для каждого привилегированного контейнера.

Третья категория — разрыв наблюдаемости (ложноположительные CRITICAL). Непривилегированные поды с RuntimeDefault выдали ложные расхождения: cri-dockerd полагается на неявный профиль Docker, который не отображается в docker inspect.

Контролируемое сравнение (Под 3 vs. Под 4)

Поды 3 и 4 имеют идентичное состояние среды выполнения; различие — декларация seccomp в поде 3. Детектор корректно выявляет расхождение только для пода 3.

Сравнение с существующими инструментами

Ни один существующий инструмент (Gatekeeper, Kyverno, Falco, Kubediff) не сопоставляет спецификацию со средой выполнения. k8s-psa занимает пустую нишу между ними.

Ограничения

Работа имеет ряд ограничений: (1) эксперименты выполнены только с Docker; в продуктивных кластерах, преимущественно использующих containerd, разрыв наблюдаемости seccomp может отсутствовать; (2) проверка seccomp основана на метаданных docker inspect, а не на статусе обеспечения на уровне ядра из /proc/<pid>/status; (3) использован одноузловой кластер, что не отражает полностью многоузловые продуктивные среды; (4) инструмент использует CLI-команды (kubectl, docker) вместо нативных API-клиентов; (5) семь тестовых подов покрывают основные измерения безопасности, но не исчерпывают все возможные конфигурации.

Практическая значимость

Комбинация privileged: true с RuntimeDefault создаёт ложное чувство безопасности. Инструменты аудита на основе API могут выдавать некорректные результаты; стандарты соответствия должны дополняться верификацией на уровне среды выполнения.

Вклад работы

Вклад: (1) инструмент k8s-psa; (2) доказательства удаления Docker профилей seccomp; (3) обнаружение разрыва наблюдаемости; (4) open-source фреймворк с 44 тестами и панелью мониторинга.

Направления дальнейших исследований

Три направления требуют дальнейшего изучения: расширение детектора верификацией через /proc/<pid>/status для устранения разрыва наблюдаемости; развёртывание на кластерах с containerd (kind, GKE, EKS) для сопоставления ландшафта верификации между средами выполнения; исследование того, создают ли мутирующие вебхуки допуска дополнительные разрывы обеспечения.

Ключевые слова: Kubernetes, безопасность контейнеров, seccomp, верификация среды выполнения, дрейф обеспечения, DevSecOps, контекст безопасности, безопасность подов.